Olympic Medal Outcome Prediction System

Project Id: AIML-5

Phase 3 Report

Name :Tanisha Rai (Team Leader)
Class Roll Number:71
University Roll Number:2028878
Section and Group:Q - G2

Name : Khushi Taneja
Class Roll Number:50
University Roll Number:2028745
Section and Group:Q - G2

TABLE OF CONTENTS

1. **Problem Description** 3
    - 1.1 Dataset Description 3
    - 1.2 Flow Diagram 4
2. **Modules for Project Implementation** 5
    - 2.1 Modules Used 5
    - 2.2 Platform Used 6
3. **Machine Learning Model / Searching Technique Used** 7
4. **Evaluation Metrics Used** 8
5. **Results and Visualizations** 9
6. **Conclusion and Future Scope** 11
7. **References** 12

1. Problem Description

The Olympic Medal Outcome Prediction System is a Python-based machine learning application developed to automate the prediction of whether an athlete will win a Gold, Silver, or Bronze medal based on their physical attributes and sport category. The system uses historical Olympic data spanning 120 years (1896–2016) to train and evaluate multiple machine learning models.

Using core concepts of Machine Learning such as data preprocessing, feature selection, classification algorithms, model evaluation, and visualization libraries, this project aims to predict medal outcomes and support data-driven insights into athletic performance patterns. The system also allows real-time prediction based on user-provided inputs.

1.1 Dataset Description

The dataset used is athlete_events.csv, a comprehensive record of Olympic athlete participation sourced from Kaggle. It contains historical data from every Olympic Games from Athens 1896 to Rio 2016.

Feature	Type	Description
ID	Integer	Unique athlete identifier
Name	String	Athlete's full name
Sex	Categorical	M (Male) or F (Female)
Age	Float	Age of the athlete
Height	Float	Height in centimeters
Weight	Float	Weight in kilograms
Team	String	Country/team represented
NOC	String	National Olympic Committee code
Games	String	Year and season of Games
Year	Integer	Olympic year
Season	Categorical	Summer or Winter
City	String	Host city
Sport	String	Sport category
Event	String	Specific event name
Medal	Categorical	Gold / Silver / Bronze / NaN (Target Variable)

Dataset Characteristics:

- Total Records: 271,116 rows × 15 columns
- Medal Distribution: Gold — 13,372 | Bronze — 13,295 | Silver — 13,116
- Sports covered: 66 unique sports including Swimming, Athletics, Gymnastics, and more

- Years covered: 1896 to 2016 (66 unique Olympic years)
- Missing values in Height and Weight handled using column mean; Medal NaN filled with 'No Medal'

1.2 Flow Diagram

The system follows a standard supervised machine learning pipeline as shown below:

Fig. 1 — System Architecture Flow Diagram

2. Modules for Project Implementation

2.1 Modules Used

The following inbuilt Python libraries were used in the implementation of the Olympic Medal Outcome Prediction System:

1. NumPy

NumPy is used for numerical computations and creating input arrays for user prediction.

Functions used:

- np.array() — To create a 2D input array from user-entered values for prediction
- np.mean() — Used indirectly via pandas fillna to compute column means

2. Pandas

Pandas is used for data manipulation, loading, and analysis of the dataset.

Functions used:

- pd.read_csv() — To load the athlete_events.csv dataset into a DataFrame
- df.head() — To display the first 5 rows of the dataset
- df.tail() — To display the last 5 rows
- df.shape — To check the number of rows and columns (271116, 15)
- df.columns — To list all column names
- df['col'].value_counts() — To count medal distribution (Gold, Silver, Bronze)
- df['col'].unique() — To view unique sports and years
- df['col'].fillna() — To fill missing Height, Weight, and Medal values
- df.dropna() — To remove remaining rows with missing values

3. Matplotlib

Matplotlib is used for creating static data visualizations.

Functions used:

- plt.plot() — To create the Total Medals over Years line chart
- plt.bar() — To create the Final Model Comparison bar chart
- plt.title() — To set chart titles
- plt.xlabel(), plt.ylabel() — To label axes
- plt.text() — To display score values on top of each bar
- plt.show() — To render and display plots
- plt.figure(figsize=()) — To set figure dimensions

4. Seaborn

Seaborn is used for advanced statistical data visualizations.

Functions used:

- sns.heatmap() — To plot the Confusion Matrix heatmap for Logistic Regression and KNN

5. Scikit-learn (sklearn)

Scikit-learn is the core machine learning library used for preprocessing, model training, and evaluation.

Modules and Functions used:

- train_test_split() — To split dataset into 80% training and 20% testing
- LabelEncoder() — To encode categorical columns: Sex, Team, Sport, Medal
- LogisticRegression() — For multi-class medal classification (max_iter=1000)
- LinearRegression() — For regression-based prediction and comparison
- KNeighborsClassifier() — For KNN-based medal classification (n_neighbors=5)
- accuracy_score() — To calculate classification accuracy
- mean_squared_error() — To calculate MSE for Linear Regression
- r2_score() — To calculate R^2 score for Linear Regression
- classification_report() — To generate precision, recall, F1-score report
- confusion_matrix() — To generate the confusion matrix

6. Warnings

Python's warnings module is used to suppress unnecessary warning messages during model training.

- warnings.filterwarnings('ignore') — To suppress convergence and deprecation warnings

2.2 Platform Used

The following platforms were used for development and implementation:

Visual Studio Code (VS Code):

Used for writing, testing, and debugging the Python code. VS Code provides a powerful development environment with extensions like Python, Jupyter, and IntelliSense that enhance the development experience.

Jupyter Notebook:

Used for interactive data analysis, model building, and visualization. Jupyter allows step-by-step execution of code cells, making it easy to explore the dataset, run models, and visualize outputs incrementally.

Python 3.12.6 (Programming Language):

Used as the core programming language. Python's extensive ecosystem of libraries (NumPy, Pandas, Scikit-learn, Matplotlib, Seaborn) makes it ideal for machine learning and data science tasks.

3. Machine Learning Model / Searching Technique Used

Three machine learning models were implemented to predict Olympic medal outcomes based on athlete attributes. Each model was trained, tested, and evaluated for comparative analysis.

1. Logistic Regression

Logistic Regression is a supervised learning classification algorithm that models the probability of a data point belonging to a particular class using the logistic (sigmoid) function. Despite its name, it is used for classification tasks rather than regression.

- Configuration: max_iter=1000 to ensure convergence on the large dataset of 271,116 records.
- Input Features: Sex, Age, Height, Weight, Team, Sport (all label encoded).
- Output: Predicted medal class — Gold, Silver, or Bronze.
- Accuracy Achieved: 85.38%
- Selected as the best-performing model based on highest accuracy.

2. Linear Regression

Linear Regression is a regression algorithm that models the relationship between input features and a continuous numerical output. In this project, it was applied on the numerically encoded Medal column (Gold=1, Silver=2, Bronze=3) for comparative analysis.

- Configuration: Default sklearn LinearRegression with no regularization.
- Evaluation: MSE and R^2 score used instead of accuracy since it is a regression model.
- MSE: 0.2803 | R^2 Score: 0.0032
- Observation: Very low R^2 score confirms that Linear Regression is unsuitable for this classification task.

3. K-Nearest Neighbors (KNN)

K-Nearest Neighbors is a non-parametric, instance-based classification algorithm. It classifies a data point based on the majority class among its K nearest neighbors in the feature space. No explicit training phase is required.

- Initial Configuration: n_neighbors=5.
- Accuracy at K=5: 83.91%
- Hyperparameter Tuning: K values from 1 to 10 were tested; accuracy increased from ~82% (K=1) to ~85% (K=10).
- Observation: KNN performs well but slightly below Logistic Regression.

Conclusion on Model Selection:

Among all three models, Logistic Regression was selected as the final model due to its highest accuracy (85.38%), faster computation, and ease of interpretation compared to KNN. Linear Regression was found unsuitable for this classification problem.

4. Evaluation Metrics Used

The following evaluation metrics were used to assess model performance in the Olympic Medal Outcome Prediction System:

1. Accuracy Score

Accuracy measures the proportion of correctly predicted instances out of the total number of instances. It is the primary metric for comparing classification models.

Formula: `Accuracy = (TP + TN) / (TP + TN + FP + FN)`

2. Mean Squared Error (MSE)

MSE measures the average of the squared differences between actual and predicted values. It is used to evaluate the Linear Regression model.

Formula: $MSE = (1/n) \times \Sigma(Actual - Predicted)^2$

3. R^2 Score (Coefficient of Determination)

R^2 Score measures the proportion of variance in the target variable that is explained by the model. A score of 1.0 means perfect prediction; 0.0 means the model explains no variance.

Formula: $R^2 = 1 - (SS_res / SS_tot)$

4. Precision

Precision measures the proportion of correctly predicted positive observations out of all predicted positives. Important when false positives must be minimized.

Formula: `Precision = TP / (TP + FP)`

5. Recall (Sensitivity)

Recall measures the proportion of actual positives that are correctly identified by the model. Important when missing a positive case is critical.

Formula: `Recall = TP / (TP + FN)`

6. F1-Score

F1-Score is the harmonic mean of Precision and Recall. It provides a balanced measure, especially useful when there is class imbalance in the dataset.

Formula: $F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$

7. Confusion Matrix

A confusion matrix is a table that compares actual vs. predicted class labels, helping visualize the per-class performance of the model.

	Predicted Positive	Predicted Negative

Actual Positive	TP (True Positive)	FN (False Negative)
Actual Negative	FP (False Positive)	TN (True Negative)

5. Results and Visualizations

5.1 Dataset Overview

The Olympic dataset was loaded and explored with the following key observations:

- Shape: 271,116 rows × 15 columns
- Unique sports: 66 (Basketball, Swimming, Athletics, Gymnastics, etc.)
- Unique years: 66 (1896 to 2016)
- Medal distribution: Gold — 13,372 | Bronze — 13,295 | Silver — 13,116
- Preprocessing: Missing Height/Weight filled with mean; Medal NaN filled with 'No Medal'; remaining NaN rows dropped

5.2 Total Medals Over Years (Line Plot)

A line chart was plotted showing total medal counts per Olympic year. The trend shows a consistent increase from the early 1900s, with peak medal counts in recent decades due to expansion of sports and events.

[Fig. 2 — Total Medals over Years (Line Chart) — paste screenshot here]

Fig. 2 — Total Medals over Years (Line Chart)

5.3 Model Results Summary

Model	Metric	Score
Logistic Regression	Accuracy	85.38%
Linear Regression	MSE	0.2803
Linear Regression	R^2 Score	0.0032
KNN (K=5)	Accuracy	83.91%

5.4 KNN Accuracy vs K Value (Line Plot)

KNN was evaluated for K values from 1 to 10. Accuracy improved steadily from ~82% at K=1 to ~85% at K=10, showing that higher K values reduce overfitting.

[Fig. 3 — KNN Accuracy vs K Value (Line Plot) — paste screenshot here]

Fig. 3 — KNN Accuracy vs K Value (Line Plot)

5.5 Logistic Regression — Confusion Matrix

The confusion matrix for Logistic Regression visualizes per-class prediction performance across Gold, Silver, and Bronze medal categories.

[Fig. 4 — Confusion Matrix: Logistic Regression (Blue Heatmap) — paste screenshot here]

Fig. 4 — Confusion Matrix: Logistic Regression (Blue Heatmap)

5.6 KNN — Confusion Matrix

The confusion matrix for KNN at K=5 shows its classification performance across all three medal categories.

[Fig. 5 — Confusion Matrix: KNN (Orange Heatmap) — paste screenshot here]

Fig. 5 — Confusion Matrix: KNN (Orange Heatmap)

5.7 Classification Reports

Classification reports were generated for both Logistic Regression and KNN showing per-class Precision, Recall, and F1-Score.

[Fig. 6 — Classification Report: Logistic Regression — paste screenshot here]

Fig. 6 — Classification Report: Logistic Regression

[Fig. 7 — Classification Report: KNN — paste screenshot here]

Fig. 7 — Classification Report: KNN

5.8 Final Model Comparison (Bar Chart)

A bar chart compares all three models. Logistic Regression achieved 0.85, KNN achieved 0.84, and Linear Regression showed a near-zero R^2 score of 0.0032, confirming it is unsuitable for this classification task.

[Fig. 8 — Final Model Comparison Bar Chart (LR=0.85, LinReg=0.0032, KNN=0.84) — paste screenshot here]

Fig. 8 — Final Model Comparison Bar Chart (LR=0.85, LinReg=0.0032, KNN=0.84)

5.9 User Input Prediction

The final system accepts real-time user input (Sex, Age, Height, Weight, Team, Sport) and predicts the likely medal outcome using the best-performing Logistic Regression model.

- Input: Sex=M, Age=25, Height=180, Weight=75, Team=India, Sport=Athletics
- Output: Predicted Medal — Gold / Silver / Bronze (based on trained model)

[Fig. 9 — User Input Prediction Output (Terminal Screenshot) — paste screenshot here]

Fig. 9 — User Input Prediction Output (Terminal Screenshot)

6. Conclusion and Future Scope

Conclusion

The Olympic Medal Outcome Prediction System was successfully developed using machine learning techniques to classify medal outcomes (Gold, Silver, Bronze) based on athlete physical attributes and sport category. Three models — Logistic Regression, Linear Regression, and K-Nearest Neighbors — were implemented and evaluated using multiple performance metrics.

Among all the models, Logistic Regression was selected as the final model with the highest accuracy of 85.38%, followed by KNN at 83.91%. Linear Regression was found unsuitable for this classification problem, with a near-zero R^2 score of 0.0032. The confusion matrix and classification report further confirmed Logistic Regression's superior performance across all medal categories.

The system also supports real-time medal prediction by taking user inputs such as age, height, weight, sex, team, and sport, and predicting the most likely medal outcome using the trained Logistic Regression model. This demonstrates the practical applicability of the system beyond academic evaluation.

Future Scope

The project can be further improved and extended in several ways:

- Advanced Models: Implement Random Forest, Support Vector Machine (SVM), or XGBoost to further improve prediction accuracy beyond the current 85.38%.
- Deep Learning: Apply neural networks (ANN/LSTM) for learning complex patterns in athlete performance data.
- Feature Expansion: Include additional features such as country GDP, training investment, historical medal counts, and event-specific statistics.
- Web / Mobile Application: Develop a user-friendly interface (web app or mobile app) where coaches and analysts can input athlete data and get real-time medal predictions.
- Real-time Data Integration: Connect to live Olympic feeds and APIs to update predictions in real-time during active Games.
- Geographical Analysis: Extend analysis to study medal-winning patterns by country, region, and decade.
- Class Imbalance Handling: Apply SMOTE or class-weighting techniques to better handle the imbalance between medal types.

7. References

Books:

1. Introduction to Machine Learning with Python – Andreas C. Muller & Sarah Guido
2. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow – Aurelien Geron
3. Let Us Python – Yashavant Kanetkar

Dataset:

- Olympic Athletes Dataset (athlete_events.csv) — Kaggle: https://www.kaggle.com/datasets/heesoo37/120-years-of-olympic-history-athletes-and-results

Online Resources:

- Scikit-learn Documentation — https://scikit-learn.org/stable/
- GeeksforGeeks — https://www.geeksforgeeks.org/
- TutorialsPoint — https://www.tutorialspoint.com/
- Stack Overflow — https://stackoverflow.com
- Matplotlib Documentation — https://matplotlib.org/
- Kaggle — https://www.kaggle.com/